

You are building a complete “brain-signal → numbers → AI model” pipeline.
I’ll walk through every task and code block slowly, in small chunks, assuming no background and low working memory.

0. Big picture of the lab

What you are doing overall:

1. Start from EEG recordings (electrical activity from the scalp).
2. Load them from CSV files into Python.
3. Clean the signals (filters).
4. Chop the continuous signals into short windows.
5. Turn each window into a list of numerical features.
6. Reduce the number of features (feature selection).
7. Train simple AI models to classify those windows.
8. Think about harder, realistic issues (different subjects, noise, robustness).

You repeat the same pattern that’s taught in the lecture:

“Pre-processing → Feature extraction → Feature selection → Classification → Evaluation”.

1. The dataset and context

What the data is

- You have EEG recordings from people doing a mental arithmetic task (e.g. repeatedly subtracting numbers like $3141 - 42$).
- Signals were already cleaned once by the original authors using ICA (eye/muscle/heart artefacts removed).
- Each subject has 1 CSV file with 19 EEG channels (Fp1, Fp2, F3, ..., Pz).
- Each file has about 30,999 rows (samples) and 19 numeric columns (channels).

Psychology angle:

Mental arithmetic increases cognitive load and changes brain rhythms in particular frequency bands (e.g. more beta activity). The dataset lets you see how “work” in the brain shows up in EEG patterns.

2. Task 1 – Loading and inspecting the dataset

Goal:

Check that the files are there, consistent, numeric, non-empty, and get a feel for scales and channels.

2.1 Imports and data directory

python

```
from pathlib import Path
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
DATA_DIR = Path("dataset")
```

- `Path("dataset")` points to the folder with all CSV EEG files.
- `numpy`, `pandas`, `matplotlib` are standard for numbers, tables, and plots.

2.2 Listing the dataset files

python

```
def list_dataset_files(data_dir: Path) -> list[Path]:
    files = sorted(data_dir.glob("s*.csv"))
    if not files:
        raise FileNotFoundError(
            f"No CSV files found in {data_dir.resolve()}. "
            "Expected files named like s00.csv ... s35.csv."
        )
    return files
```

- Looks for files named like `s00.csv`, `s01.csv`, ... under `dataset/`.
- If none are found, it raises an error with a helpful message.

Then:

python

```
files = list_dataset_files(DATA_DIR)

print(f"Found {len(files)} subject files in:
{DATA_DIR.resolve()}")

print("First 5 files:", [f.name for f in files[:5]])
print("Last 5 files:", [f.name for f in files[-5:]])
```

- Confirms there are 36 subject files (`s00.csv` to `s35.csv`).

2.3 Reading each CSV safely

python

```
def read_csv_safely(path: Path) -> pd.DataFrame:
    try:
        df = pd.read_csv(path)
    except Exception as e:
        raise RuntimeError(f"Failed to read {path.name}: {e}")
    from e
    if df.empty:
        raise ValueError(f"{path.name} loaded but contains no
rows.")
    return df
```

- Tries to load the CSV into a pandas DataFrame.
- If reading fails, it raises a clear error.

- If the file is empty, it raises another clear error.
This guards against silent failures later.

2.4 Summarising each DataFrame

python

```
def summarise_dataframe(df: pd.DataFrame) -> dict:
    n_rows, n_cols = df.shape
    missing = int(df.isna().sum().sum())
    numeric_cols =
df.select_dtypes(include=[np.number]).columns.tolist()
    non_numeric_cols = [c for c in df.columns if c not in
numeric_cols]
    return {
        "rows": n_rows,
        "cols": n_cols,
        "missing_values": missing,
        "numeric_cols": len(numeric_cols),
        "non_numeric_cols": len(non_numeric_cols),
        "columns": df.columns.tolist(),
    }
```

- Counts:
 - number of rows and columns,
 - how many missing values,
 - how many numeric vs non-numeric columns,
 - saves the column names.

Loop:

python

```
summaries = []
```

```

dataframes = {}
for f in files:
    df = read_csv_safely(f)
    dataframes[f.stem] = df
    s = summarise_dataframe(df)
    s["file"] = f.name
    s["subject"] = f.stem
    summaries.append(s)

summary_df = pd.DataFrame(summaries).sort_values("subject")
display(summary_df[["subject", "rows", "cols", "numeric_cols",
                    "non_numeric_cols", "missing_values"]])

```

What you learn:

- Every subject has 30,999 rows, 19 columns, all numeric, no missing values.
- Good: data is clean and consistent in size.

2.5 Inferring EEG channels

python

```

def infer_channel_columns(df: pd.DataFrame) -> list[str]:
    numeric_cols =
df.select_dtypes(include=[np.number]).columns.tolist()
    exclude = {"time", "timestamp", "ts", "label", "class",
"event",
                "subject", "id"}
    channels = [c for c in numeric_cols if c.strip().lower() not
in exclude]
    return channels

```

- A simple rule:
 - treat numeric columns as potential EEG channels,
 - skip typical metadata names like “time”, “label”.
- Here all 19 columns are numeric and not excluded, so all 19 are treated as channels.

2.6 Checking column schemas

python

```
all_columns = summary_df["columns"].tolist()
unique_schemas = {tuple(cols) for cols in all_columns}
print(f"Unique column schemas: {len(unique_schemas)}")
```

- Compares column *names* across files.
- It reports `Unique column schemas: 36`, so each file has different column names.

Why?

The original conversion from EDF to CSV used numeric amplitudes as headers, so each file’s column names are weird numbers, but the *structure* (19 numeric columns) is the same.

They warn you but still treat all numeric columns as channels, so you can proceed.

2.7 Inspecting one subject and plotting

python

```
example_subject = summary_df.iloc[0]["subject"]
df0 = dataframes[example_subject]
print(f"\nExample subject: {example_subject} shape={df0.shape}")
display(df0.head())
channels = infer_channel_columns(df0)
print(f"Candidate EEG channels ...: {len(channels)}")
```

```
print(channels[:25], "... " if len(channels) > 25 else "")
```

- Chooses `s00` as example.
- Shows first few rows (raw numbers).
- Confirms 19 channels.

Plot a few channels:

```
python
```

```
plot_channels = channels[:5]
n_samples = min(2000, len(df0))
x = np.arange(n_samples)
plt.figure(figsize=(12, 6))
for c in plot_channels:
    plt.plot(x, df0[c].values[:n_samples], label=c)
...
```

- Draws the first 2000 samples of up to 5 channels.
- Lets you visually check that signals look like continuous EEG (wiggly lines), not flat or obviously broken.

2.8 Basic distribution summary and scale

```
python
```

```
desc = df0[plot_channels].describe(
    percentiles=[0.01, 0.05, 0.5, 0.95, 0.99]
).T
display(desc)
```

- Gives per-channel stats: mean, std, min, max, and some percentiles.

- You see values around tens of microvolts (or some arbitrary unit), with wide ranges.

Per-subject scale:

python

```
scale_rows = []
for subj, df in dataframes.items():
    ch = infer_channel_columns(df)
    if not ch:
        continue
    vals = df[ch].values
    med = np.nanmedian(vals, axis=0)
    mad = np.nanmedian(np.abs(vals - med), axis=0)
    scale_rows.append({
        "subject": subj,
        "n_channels": len(ch),
        "median_mad": float(np.nanmedian(mad)),
        "median_std": float(np.nanmedian(np.nanstd(vals,
axis=0))),
    })
scale_df = pd.DataFrame(scale_rows).sort_values("subject")
display(scale_df.head(10))
```

- Median absolute deviation (MAD) and std per subject tell you typical amplitude scale.
- Values are similar across subjects, which is good for later normalisation.

Task 1 outcome:

You know:

- how many files, their shape,

- that data is numeric and non-empty,
 - approximate signal amplitude,
 - candidate channels,
- so you're ready to choose window size and features.
-

3. Task 2 – Pre-processing: filtering and windowing

Goal:

Clean the raw continuous EEG and cut it into equal chunks (windows) for machine learning.

3.1 Configuration parameters

python

```
FS = 256          # Sampling frequency (Hz)
LOWCUT = 1.0      # Band-pass lower cutoff
HIGH CUT = 40.0   # Band-pass upper cutoff
NOTCH_FREQ = 50.0 # Power-line frequency
Q_FACTOR = 30.0   # Notch filter quality
WINDOW_SEC = 1.0  # Window length in seconds
```

- Assumes samples are taken at 256 Hz (256 points per second).
- Keeps only 1–40 Hz (most EEG bands of interest) with a band-pass filter.
- Applies a notch at 50 Hz to reduce mains interference.
- Uses 1-second windows (256 samples each).

3.2 Filter design functions

Band-pass:

python

```
def bandpass_filter(data, fs, lowcut, highcut, order=4):
    nyq = 0.5 * fs
```

```

low = lowcut / nyq
high = highcut / nyq
b, a = butter(order, [low, high], btype="band")
return filtfilt(b, a, data, axis=0)

```

- `nyq` is the Nyquist frequency (half of sampling rate).
- Makes a 4th-order Butterworth band-pass filter in Hz range.
- `filtfilt` applies filter forward and backward → zero phase distortion.

Notch:

python

```

def notch_filter(data, fs, freq, q):
    nyq = 0.5 * fs
    w0 = freq / nyq
    b, a = iirnotch(w0, q)
    return filtfilt(b, a, data, axis=0)

```

- Designs a narrow notch at 50 Hz, with quality factor Q=30.
- Helps remove power-line hum.

3.3 Select one subject and apply filters

python

```

subject_id = list(dataframes.keys())[0] # e.g. 's00'
df = dataframes[subject_id]
channels = infer_channel_columns(df)
eeg = df[channels].values
print(f"Subject: {subject_id}")
print(f"EEG shape (samples × channels): {eeg.shape}")

```

- Extracts a NumPy array of shape (30999, 19).

Apply filters:

python

```
eeg_bandpassed = bandpass_filter(eeg, FS, LOWCUT, HIGHCUT)
eeg_filtered = notch_filter(eeg_bandpassed, FS, NOTCH_FREQ,
Q_FACTOR)
```

- First removes everything outside 1–40 Hz.
- Then reduces 50 Hz noise.

3.4 Visual check: raw vs filtered

python

```
n_plot = min(2000, eeg.shape[0])
ch_idx = 0
plt.figure(figsize=(12, 6))
plt.plot(eeg[:n_plot, ch_idx], label="Raw", alpha=0.6)
plt.plot(eeg_filtered[:n_plot, ch_idx], label="Filtered",
linewidth=2)
...
```

- Plots the same channel before and after filtering.
- You should see the filtered signal is smoother and with less noise at high frequency.

3.5 Segment into fixed windows

python

```
window_size = int(WINDOW_SEC * FS)           # 256 samples
```

```

n_windows = eeg_filtered.shape[0] // window_size
eeg_windows = eeg_filtered[:n_windows * window_size]
eeg_windows = eeg_windows.reshape(
    n_windows, window_size, eeg_filtered.shape[1]
)
print(f"Segmented EEG shape: {eeg_windows.shape}")

```

- Takes as many full 1-second windows as possible.
- For s00: $30999 // 256 = 121$ windows.
- `eeg_windows` shape is (121, 256, 19):
 - 121 windows,
 - each 256 time samples (1 second),
 - 19 channels.

Task 2 outcome:

A clean, filtered, windowed 3D array ready for feature extraction.

4. Task 3 – Feature extraction

Goal:

Turn each 1-second window into a single row of numbers (features) that describe the signal in time, frequency, and time-frequency domains.

4.1 Frequency bands

python

```

BANDS = {
    "delta": (0.5, 4.0),
    "theta": (4.0, 8.0),
    "alpha": (8.0, 13.0),
    "beta": (13.0, 30.0),
}

```

- Standard EEG bands, defined as frequency ranges in Hz.

4.2 Hjorth parameters

python

```
def hjorth_parameters(x: np.ndarray) -> tuple[float, float, float]:
    x = np.asarray(x)
    dx = np.diff(x)
    ddx = np.diff(dx)
    var_x = np.var(x)
    var_dx = np.var(dx) if dx.size else 0.0
    var_ddx = np.var(ddx) if ddx.size else 0.0
    activity = var_x
    mobility = np.sqrt(var_dx / var_x) if var_x > 0 else 0.0
    mobility_dx = np.sqrt(var_ddx / var_dx) if var_dx > 0 else 0.0
    complexity = (mobility_dx / mobility) if mobility > 0 else 0.0
    return activity, mobility, complexity
```

For a 1D signal x (one channel, one window):

- Activity $\approx$ variance (overall power).
- Mobility $\approx$ average frequency (how quickly it changes).
- Complexity $\approx$ how the frequency pattern itself changes (irregularity).

These summarise dynamics in 3 numbers per channel per window.

4.3 Band powers from PSD (Welch)

python

```
from scipy.signal import welch
```

```
def bandpowers_from_psd(x: np.ndarray, fs: float, bands: dict)
-> dict:
    f, pxx = welch(x, fs=fs, nperseg=min(256, len(x)))
    out = {}
    for name, (fmin, fmax) in bands.items():
        idx = np.logical_and(f >= fmin, f <= fmax)
        out[name] = np.trapezoid(pxx[idx], f[idx]) if
np.any(idx) else 0.0
    return out
```

- `welch` estimates the power spectral density (PSD) of the signal.
- For each band (delta, theta, alpha, beta) it integrates PSD over that frequency range (using `np.trapezoid`).
- Returns something like `{"alpha": 12.3, "beta": 3.4, ...}`.

4.4 Time–frequency summary via STFT

python

```
from scipy.signal import stft
```

```
def stft_summary_features(x: np.ndarray, fs: float) -> dict:
    f, t, Zxx = stft(x, fs=fs, nperseg=min(128, len(x)))
    mag = np.abs(Zxx)
    return {
        "stft_mag_mean": float(np.mean(mag)),
        "stft_mag_std": float(np.std(mag)),
    }
```

```

    "stft_mag_max": float(np.max(mag)),
}

```

- STFT chops the signal into mini-windows and does a Fourier transform for each.
- `mag` is the magnitude of complex values (how strong each frequency is at each time).
- You summarise that huge matrix with 3 basic stats: mean, std, max.

4.5 Extracting features for one window

python

```

def extract_features_for_window(window: np.ndarray, fs: float,
                                channel_names: list[str]) ->
dict:
    feats = {}
    for ch_i, ch_name in enumerate(channel_names):
        x = window[:, ch_i]
        # Time-domain
        feats[f"{ch_name}_mean"] = float(np.mean(x))
        feats[f"{ch_name}_var"] = float(np.var(x))
        feats[f"{ch_name}_std"] = float(np.std(x))
        # Hjorth
        act, mob, comp = hjorth_parameters(x)
        feats[f"{ch_name}_hjorth_activity"] = float(act)
        feats[f"{ch_name}_hjorth_mobility"] = float(mob)
        feats[f"{ch_name}_hjorth_complexity"] = float(comp)
        # Band powers
        bp = bandpowers_from_psd(x, fs, BANDS)
        for band_name, power in bp.items():

```

```

        feats[f"{ch_name}_bp_{band_name}"] = float(power)
# STFT summaries
tf = stft_summary_features(x, fs)
for k, v in tf.items():
    feats[f"{ch_name}_{k}"] = float(v)
return feats

```

For each channel in that window:

- 3 time stats (mean, var, std).
- 3 Hjorth parameters.
- 4 band powers (delta, theta, alpha, beta).
- 3 STFT summaries.

So per channel: $3 + 3 + 4 + 3 = 13$ features.

For 19 channels: $19 \times 13 = 247$ features per window (matches output).

4.6 Applying to all windows

python

```

feature_rows = []
for w in range(eeg_windows.shape[0]):
    feature_rows.append(
        extract_features_for_window(eeg_windows[w], FS,
        channels)
    )

features_df = pd.DataFrame(feature_rows)
print("Feature matrix shape (windows × features):",
features_df.shape)
display(features_df.head())

```


- For 121 windows, you get a DataFrame of shape `(121, 247)`.
- Each row = one 1-second window.
- Each column = one named feature (channel + feature type).

Task 3 outcome:

You have a classic machine-learning feature matrix `x`: rows=windows, columns=features.

5. Task 4 – Feature selection

Goal:

You now have 247 features. That's quite a lot considering you have only 121 windows. Here you:

- Create labels (targets) for a simple classification problem.
- Train a baseline model with all features.
- Try 3 feature-selection strategies: filter, wrapper, embedded.
- Compare performance.

5.1 Creating a simple label `y` (high vs low alpha power)

There is no original label in the single-subject data for Task 4.

So they invent a proxy label: is the average alpha-band power high or low in this window?

python

```
from scipy.signal import welch
```

```
if "y" not in globals():
    if "eeg_windows" not in globals():
        raise ValueError(...)
    if "FS" not in globals():
        raise ValueError(...)
```

```
alpha_lo, alpha_hi = 8.0, 13.0
```

```

alpha_powers = []
for w in range(eeg_windows.shape[0]):
    win = eeg_windows[w]  # shape: (window_size, n_channels)
    ch_p = []
    for ch in range(win.shape[1]):
        f, pxx = welch(win[:, ch], fs=FS, nperseg=min(256,
win.shape[0]))
        idx = (f >= alpha_lo) & (f <= alpha_hi)
        bp = np.trapezoid(pxx[idx], f[idx]) if np.any(idx)
    else 0.0
        ch_p.append(bp)
    alpha_powers.append(np.mean(ch_p))
alpha_powers = np.asarray(alpha_powers)
thr = np.median(alpha_powers)
y = (alpha_powers > thr).astype(int)
print("y created as proxy label: high vs low alpha power
(median split)")
print("Class balance:", np.bincount(y))

```

- Compute alpha power per channel per window, then average across channels → one alpha power per window.
- Use the median as threshold:
 - $y=1$ if $\alpha > \text{median}$ (high alpha),
 - $y=0$ otherwise (low alpha).
- Gives 61 windows in class 0, 60 in class 1 → nicely balanced.

This is not a “real” BCI task, but is good enough to demonstrate feature selection mechanics.

5.2 Evaluation helper and baseline classifier

python

```

X = features_df.values
feature_names = features_df.columns.to_list()

from sklearn.model_selection import StratifiedKFold,
cross_val_score
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import make_scorer, balanced_accuracy_score
from sklearn.linear_model import LogisticRegression

bal_acc = make_scorer(balanced_accuracy_score)
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

```

- `StratifiedKFold`: splits the data into 5 folds keeping class ratios similar in each fold.
- `balanced_accuracy_score`: average of per-class recall; works better when classes might be imbalanced.

Helper:

python

```

def evaluate_model(X_in, y_in, estimator, cv):
    scores = cross_val_score(estimator, X_in, y_in, cv=cv,
                             scoring=bal_acc, n_jobs=-1)
    return float(scores.mean()), float(scores.std())

```

- Runs cross-validation and returns mean and std of balanced accuracy.

Baseline model:

python

```
baseline_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=2000, n_jobs=-1))
])
base_mean, base_std = evaluate_model(X, y, baseline_clf, cv)
print(f"Baseline (all features) balanced accuracy:
{base_mean:.3f} ± {base_std:.3f}")
```

- `StandardScaler` standardises features (mean 0, std 1) → important for logistic regression.
- Baseline balanced accuracy $\approx 0.867 \pm 0.049$ using all 247 features.

5.3 Filter method – Mutual Information (MI)

python

```
from sklearn.feature_selection import SelectKBest,
mutual_info_classif

k = min(200, X.shape[1]) # 200
filter_selector = SelectKBest(score_func=mutual_info_classif,
k=k)
filter_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("select", filter_selector),
    ("clf", LogisticRegression(max_iter=2000, n_jobs=-1))
])
f_mean, f_std = evaluate_model(X, y, filter_clf, cv)
print(f"Filter (MI top-{k}) balanced accuracy: {f_mean:.3f} ±
{f_std:.3f}")
```

- `mutual_info_classif`: measures how much information each feature gives about the class label, one feature at a time.
- `SelectKBest`: keeps top 200 features by MI.
- Performance is $\approx 0.859 \pm 0.034$, slightly below baseline.

List selected features:

python

```
filter_selector.fit(StandardScaler().fit_transform(X), y)
mask = filter_selector.get_support()
selected_filter_features = [n for n, m in zip(feature_names,
mask) if m]
print(f"Filter selected features:
{len(selected_filter_features)}")
```

- Confirms 200 features selected.

5.4 Wrapper method – RFE (Recursive Feature Elimination)

python

```
from sklearn.feature_selection import RFE

n_select = min(50, X.shape[1])
rfe_estimator = LogisticRegression(max_iter=2000, n_jobs=-1)
wrapper_selector = RFE(estimator=rfe_estimator,
                        n_features_to_select=n_select, step=0.1)
wrapper_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("select", wrapper_selector),
    ("clf", LogisticRegression(max_iter=2000, n_jobs=-1))
```

```

])
w_mean, w_std = evaluate_model(X, y, wrapper_clf, cv)
print(f"Wrapper (RFE {n_select}) balanced accuracy: {w_mean:.3f}
± {w_std:.3f}")

```

- RFE:
 - Starts with all features.
 - Trains logistic regression.
 - Removes least important ~10% features in each step.
 - Repeats until only 50 remain.
- Balanced accuracy $\approx 0.868 \pm 0.049$ (similar to baseline but with fewer features).

Selected features:

```

python

wrapper_selector.fit(StandardScaler().fit_transform(X), y)
mask = wrapper_selector.get_support()
selected_wrapper_features = [n for n, m in zip(feature_names,
mask) if m]

print(f"Wrapper selected features:
{len(selected_wrapper_features)}")

```

- Confirms 50 features kept.

5.5 Embedded method – Random Forest importance

```

python

from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(
    n_estimators=300,
    random_state=42,

```

```

    n_jobs=-1,
    class_weight="balanced_subsample"
)
X_scaled = StandardScaler().fit_transform(X)
rf.fit(X_scaled, y)
importances = rf.feature_importances_
top_k = min(100, len(importances))
top_idx = np.argsort(importances)[::-1][:top_k]
X_rf = X[:, top_idx]

```

- Random Forest trains 300 decision trees.
- `feature_importances_` tells how useful each feature is across all splits in all trees.
- Keep the best 100 features by importance (`top_idx`).

Then evaluate with logistic regression on these 100 features:

python

```

rf_eval_clf = Pipeline([
    ("scaler", StandardScaler()),
    ("clf", LogisticRegression(max_iter=2000, n_jobs=-1))
])
e_mean, e_std = evaluate_model(X_rf, y, rf_eval_clf, cv)
print(f"Embedded (RF importance top-{top_k}) balanced accuracy:
{e_mean:.3f} ± {e_std:.3f}")

```

- Balanced accuracy $\approx 0.951 \pm 0.017$ – clearly best among the four setups.

Summary table:

python

```

results = pd.DataFrame([
    {"method": "All features", ...},
    {"method": f"Filter (MI top-{k})", ...},
    {"method": f"Wrapper (RFE {n_select})", ...},
    {"method": f"Embedded (RF top-{top_k})", ...},
]).sort_values("bal_acc_mean", ascending=False)
display(results)

```

You see:

- RF top-100 features: ~0.951
- RFE 50: ~0.868
- All 247: ~0.867
- MI top-200: ~0.859

Task 4 outcome:

Feature selection can:

- keep or slightly improve performance with fewer features,
 - or, if done well (RF top-k), improve both performance and robustness.
-

6. Task 5 – Classification models and confusion matrices

Goal:

Compare multiple models (LDA, Logistic Regression, Random Forest) on different feature sets and inspect confusion matrices.

6.1 Prepare feature sets

python

```

X_all = features_df.values
feature_sets = {
    "All features": X_all

```



```

}

if "top_idx" in globals():
    feature_sets["RF top-k features"] = X_all[:, top_idx]
print("Available feature sets:", list(feature_sets.keys()))

```

- Two feature sets:
 - All 247 features.
 - Only the 100 best features by RF importance.

6.2 Define models

python

```

from sklearn.discriminant_analysis import
LinearDiscriminantAnalysis

```

```

models = {
    "LDA": Pipeline([
        ("scaler", StandardScaler()),
        ("clf", LinearDiscriminantAnalysis())
    ]),
    "Logistic Regression": Pipeline([
        ("scaler", StandardScaler()),
        ("clf", LogisticRegression(max_iter=2000, n_jobs=-1))
    ]),
    "Random Forest": Pipeline([
        ("clf", RandomForestClassifier(
            n_estimators=300,
            random_state=42,

```

```

        n_jobs=-1,
        class_weight="balanced_subsample"
    ))
l)
}

```

- LDA: linear classifier that finds a projection that best separates the two classes.
- Logistic Regression: linear probabilistic classifier.
- Random Forest: ensemble of trees, handles non-linear relationships.

6.3 Cross-validation and confusion matrices

python

```

from sklearn.model_selection import StratifiedKFold,
cross_val_predict

```

```

from sklearn.metrics import balanced_accuracy_score,
confusion_matrix, ConfusionMatrixDisplay

```

```

cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

```

```

results = []

```

```

for fs_name, X in feature_sets.items():

```

```

    for model_name, model in models.items():

```

```

        y_pred = cross_val_predict(model, X, y, cv=cv,
n_jobs=-1)

```

```

        bal_acc = balanced_accuracy_score(y, y_pred)

```

```

        results.append({

```

```

            "feature_set": fs_name,

```

```

            "model": model_name,

```

```

        "balanced_accuracy": bal_acc
    })
    cm = confusion_matrix(y, y_pred)
    disp = ConfusionMatrixDisplay(confusion_matrix=cm)
    plt.figure(figsize=(6, 6))
    disp.plot(values_format="d")
    plt.title(f"Confusion Matrix: {model_name} ({fs_name})")
    plt.tight_layout()
    plt.show()

```

- `cross_val_predict` gives you predictions for every sample as if it were test data.
- `confusion_matrix` counts:
 - True 0 predicted 0,
 - True 0 predicted 1,
 - True 1 predicted 0,
 - True 1 predicted 1.
- Balanced accuracy computed again.

Summary:

python

```

results_df = pd.DataFrame(results).sort_values(
    ["feature_set", "balanced_accuracy"],
    ascending=[True, False]
)
display(results_df)

```

You get roughly:

Feature set	Model	Balanced accuracy
All features	Logistic Regression	0.868
All features	Random Forest	0.860
All features	LDA	0.785
RF top-k features	Logistic Regression	0.951
RF top-k features	Random Forest	0.876
RF top-k features	LDA	0.528

Interpretation:

- With all features, logistic regression slightly beats RF here.
- With RF top-k features, logistic regression becomes very strong (≈ 0.95).
- LDA struggles on RF-selected features (they're not chosen for LDA specifically).

This illustrates:

- Model performance depends on both feature set and algorithm.
- Ensemble-based feature selection + linear classifier can be powerful.

7. Tasks 6 & 7 – Concepts (not fully coded here)

These are “apply your knowledge” tasks: think about more realistic, harder scenarios.

7.1 Task 6 – Cross-subject generalisation

Idea:

Models trained on one subject often fail on other subjects because brain signals differ between people.

You should:

- Segment data for *all* subjects and keep `subject_id` for each window.
- Do Leave-One-Subject-Out (LOSO) cross-validation:
 - Pick one subject as test.
 - Train on all other subjects.
 - Evaluate on this held-out subject.
 - Repeat for each subject.
- Compute balanced accuracy per subject and the mean $\pm$ variance across subjects.
- Compare this to standard within-subject k-fold CV (like what you already did).

This shows “domain shift”: performance drops when test data differs (new person) from training data.

7.2 Task 7 – Feature stability and model robustness

Idea:

High accuracy once is not enough. You want features and models that are:

- Stable across cross-validation folds.
- Robust to:
 - different window lengths (0.5s, 1s, 2s),
 - added noise,
 - randomness in training.

Things to try:

- Repeat feature selection (e.g. MI or RF importance) across different folds and measure:
 - How often the same features are chosen (top-k overlap).
 - Rank correlation between feature importance lists.
- Re-run classification with:
 - different `WINDOW_SEC` values,
 - Gaussian noise added to EEG windows at various SNRs.
- Compare how much performance drops for:

- simple linear models (LDA, Logistic Regression),
- Random Forest (ensemble).

This builds intuition about which models are more robust to real-world messiness.

8. How to mentally “chunk” what you learned

For low working memory, keep this picture:

1. Data loading (Task 1)
 - Many subjects, each file is a big table of 19 EEG channels over time.
2. Clean + window (Task 2)
 - Filters (1–40 Hz band-pass, 50 Hz notch).
 - Chop into 1-second windows: shape `(n_windows, 256, 19)`.
3. Describe each window (Task 3)
 - Time stats, Hjorth, band powers, STFT summaries → about 247 features per window.
4. Select useful features (Task 4)
 - Create a simple label (high vs low alpha).
 - Try MI, RFE, RF importance.
 - RF top-100 + logistic regression works best.
5. Train and compare models (Task 5)
 - LDA, Logistic Regression, Random Forest.
 - Check balanced accuracy and confusion matrices.
6. Think about reality (Tasks 6–7)
 - New subjects (domain shift).
 - Window length, noise, feature stability.